

MATH LIB5

An Immitable System of Safety Boundaries
for Verified Symbolic Compute

0

/ x

/

+

*
* +

#

#

+

0

x

*

#

MATHLIB5

An Immutable System of Safety Boundaries for Verified Symbolic Compute

Ahmad Ali Parr · SnapKitty Collective · SNAPKITTYWEST
Simplified Edition — for engineers, reviewers, and the curious

1. What MATHLIB5 Is

MATHLIB5 is a system that **computes summations and matrix algebra** while proving, at every step, that the answer is correct. You write a small APL expression; the system transforms it through ten checked stages and produces both a fast C program *and* a machine-checked certificate that the program does exactly what the math says.

The point is not speed. The point is **trust you can verify**. Every artifact the system emits carries a receipt, and every receipt is sealed into an append-only chain anchored to the Bitcoin ledger.

Why "safety boundaries"?

We think of the system as a sequence of doors. To pass through a door, an artifact must satisfy a contract. If it does, the door stamps a receipt and lets it through. If it does not, the build stops. No unverified artifact can silently slip through — the system is *fail-loud* by design.

2. The Big Idea in One Paragraph

A summation like $1+2+3+4$ has a closed form: $n(n+1)/2$. MATHLIB5 proves that closed form inside the Lean proof assistant, then generates a C program that computes the same thing. It checks that the generated program matches the proof, signs the result, and writes the signature to a chain that the public Bitcoin network timestamps. A stranger on another continent can confirm the whole story without trusting us.

3. The Pipeline in Plain Language

1. APL frontend	You write the numeric expression in APL, a concise array language.
2. Intermediate representation	The expression becomes a typed, shape-aware tree the rest of the system can reason about.
3. Liquid Haskell	A refinement type checker proves the values stay inside their promised bounds.
4. Lean 4	The closed-form mathematical identity is proven as a theorem.
5. C-- emission	The verified expression is turned into low-level C-- code.

6. Static analysis	The code is checked for out-of-bounds reads and uninitialized values.
7. Policy / FOL	The correctness claim is written as a first-order-logic formula.
8. P/NP swarm	Open proof gaps are posted as solvable problems that agents compete to close.
9. WORM ledger	Every receipt is appended to a write-once chain that cannot be rewritten.
10. Completeness	A score reports how much of the system is actually proven, honestly.

Each layer is a **safety boundary**. The output contract of one becomes the input contract of the next, so a break at any single layer is caught before it can propagate.

4. The Verified Kernel

At the core sits a small verified kernel (written in a Coq-like language) that stores a polynomial as a tagged value and reduces it by the rules of arithmetic. Because the kernel is tiny and proven, the rest of the system can rely on it the way a building relies on its foundation. A foreign-function bridge then lets this verified kernel call ordinary C code and Lean code across a single, audited boundary.

5. Provenance You Can Check

MATHLIB5 does not ask you to trust a server. It publishes three things you can verify yourself:

- **SHA-256 fingerprint** of the published document and the repository receipt.
- **Ed25519 signature** (the "Plasma Gate") over that fingerprint, using a published public key.
- **Bitcoin OP_RETURN** message embedding the fingerprint on the public ledger, giving a tamper-resistant timestamp.

Fingerprint (SHA-256):

```
88313c7cea5c462eec80069f12fc28d771465c3aebeec2a79f4661d502a03491
```

Ed25519 public key:

```
18d816694de0deae621e913177bdfa3547e5d4cc2f9d91dfdcc3a16d03d02141
```

Bitcoin OP_RETURN payload:

```
MATH5:88313c7cea5c462eec80069f12fc28d771465c3aebeec2a79f4661d502a03491 (70 bytes)
```

6. Security and Honesty

If an attacker tampered with an intermediate artifact, they would have to forge an Ed25519 signature (assumed hard) or rewrite the Bitcoin anchor (economically expensive). Even a fully compromised build machine cannot produce a fake receipt that passes the public check. And because unproven lemmas are recorded as open debt rather than hidden, you always know exactly how much of the system rests on certified ground.

7. How to Verify It Yourself

Clone the repository and run the built-in verification gate:

```
git clone https://github.com/SNAPKITTYWEST/mathlib5
cd mathlib5 && npm ci
npm run verify:all
python paper/mathlib5/full/anchors/verify_anchors.py
```

The last command checks the fingerprint, the signature, and the OP_RETURN anchor, and prints **VERIFIED** when all three agree.

8. A Note on Scope

MATHLIB5 is deliberately narrow: it certifies summation and matrix contraction for a small APL fragment. It is not trying to replace general mathematical libraries or production C compilers. Within its domain, however, it is fully certified end-to-end, and that certificate is publicly anchorable.

9. Glossary

Closed form	A direct formula (e.g. $n(n+1)/2$) instead of a step-by-step sum.
Receipt	A signed record that a boundary was passed.
WORM	Write-Once-Read-Many: a ledger that can only be appended to.
Plasma Gate	The Ed25519 signing authority that seals receipts.
OP_RETURN	A Bitcoin transaction field used to embed arbitrary data.
P/NP swarm	A crowd of agents that compete to solve and verify open proofs.

Repository: github.com/SNAPKITTYWEST/mathlib5 · ORCID: 0009-0006-1916-5245